

Cheat Sheet — Programmazione ed Algoritmica

3ª Prova in Itinere · 13 marzo 2026 · Diego Stefanini

Heap e HeapSort

Definizioni

Heap binario: albero binario quasi completo (tutti i livelli pieni tranne l'ultimo, riempito da sx a dx) con proprietà di heap:

- **Max-Heap:** $A[\text{Parent}(i)] \geq A[i]$ per ogni nodo i
- **Min-Heap:** $A[\text{Parent}(i)] \leq A[i]$ per ogni nodo i

Proprietà strutturali (max-heap $A[1..n]$):

- Massimo in $A[1]$
- Altezza: $h = \lfloor \log_2 n \rfloor = \Theta(\log n)$
- Foglie: posizioni $\lfloor n/2 \rfloor + 1, \dots, n$
- Nodi ad altezza h' : al più $\lfloor n/2^{h'+1} \rfloor$

Rappresentazione come array

Radice in $A[1]$. Per il nodo in posizione i :

$$\text{Parent}(i) = \left\lfloor \frac{i}{2} \right\rfloor \quad \text{Left}(i) = 2i \quad \text{Right}(i) = 2i + 1$$

Attributi: $A.\text{length}$ (dimensione array), $A.\text{heap-size}$ (elementi validi dell'heap).

Max-Heapify — $O(\log n)$

Ripristina proprietà max-heap nel sottoalbero radicato in i , assumendo che i sottoalberi figli siano già max-heap. Confronta $A[i]$ con i figli, scambia col maggiore, ricorre verso il basso (sift-down).

```
maxHeapify(A, i, heapsize):
    l = left(i), r = right(i), largest = i
    if l <= heapsize && A[l] > A[i]: largest = l
    if r <= heapsize && A[r] > A[largest]: largest = r
    if largest != i:
        swap(A[i], A[largest])
        maxHeapify(A, largest, heapsize)
```

Complessità: $O(h)$ dove h è altezza del nodo. Ricorrenza: $T(n) \leq T(2n/3) + \Theta(1) \Rightarrow O(\log n)$.

Correttezza: induzione sull'altezza h . Base ($h = 0$): foglia, nulla da fare. Passo: se $\text{largest} = i$ ok; altrimenti dopo lo scambio il sottoalbero violante ha altezza $h - 1$, per ipotesi induttiva la ricorsione corregge.

Build-Max-Heap — $\Theta(n)$

Trasforma array arbitrario in max-heap. Applica Max-Heapify dal basso verso l'alto, partendo da $\lfloor n/2 \rfloor$ (le foglie sono già heap banali).

```
buildMaxHeap(A, n):
    heapsize = n
    for i = n/2 downto 1:
        maxHeapify(A, i, heapsize)
```

Invariante: all'inizio di ogni iterazione, ogni nodo $j \in \{i + 1, \dots, n\}$ è radice di un max-heap.

Analisi complessità:

- Ingenua: $O(n) \times O(\log n) = O(n \log n)$ (non stretta)
- Stretta: $\sum_{h'=0}^{\lfloor \log n \rfloor} \lfloor n/2^{h'+1} \rfloor \cdot O(h') = O\left(n \sum h' / 2^{h'}\right) = O(2n) = \Theta(n)$
- Intuizione: la maggior parte dei nodi ha altezza piccola ($n/2$ foglie, $n/4$ altezza 1, ...)

HeapSort — $\Theta(n \log n)$

1. buildMaxHeap(A, n) — massimo in $A[1]$
2. Scambia $A[1]$ con $A[\text{heapsize}]$, decrementa heapsize
3. maxHeapify($A, 1, \text{heapsize}$) per ripristinare
4. Ripeti finché heapsize = 1

```
heapsort(A, n):
    buildMaxHeap(A, n)
    heapsize = n
    for i = n downto 2:
        swap(A[1], A[i])
        heapsize = heapsize - 1
        maxHeapify(A, 1, heapsize)
```

Invariante: all'inizio di ogni iterazione: $A[1..i]$ è max-heap con gli i elementi più piccoli; $A[i + 1..n]$ contiene gli $n - i$ più grandi, ordinati.

Complessità: $\Theta(n) + (n - 1) \cdot O(\log n) = \Theta(n \log n)$ (pessimo, ottimo, medio)

In-place: sì (memoria $O(1)$ extra) · **Stabile:** no

Code di priorità (max)

Operazioni: Maximum, Extract-Max, Increase-Key, Insert.

Complessità con max-heap:

Maximum(A)	$O(1)$ — restituisce $A[1]$
ExtractMax(A)	$O(\log n)$ — swap $A[1] \leftrightarrow A[n]$, heapify
IncreaseKey(A, i, k)	$O(\log n)$ — sift-up
Insert(A, k)	$O(\log n)$ — aggiungi — ∞ , increase-key

Extract-Max: salva $A[1]$, poni $A[1] := A[\text{heapsize}]$, decrementa heapsize, maxHeapify($A, 1, \text{heapsize}$).

Increase-Key (sift-up): poni $A[i] := k$, risalisci scambiando con il padre finché $A[\text{Parent}(i)] \geq A[i]$.

```
heapIncreaseKey(A, i, key):
    A[i] = key
    while i > 1 && A[parent(i)] < A[i]:
        swap(A[i], A[parent(i)])
        i = parent(i)
```

Invariante Increase-Key: l'array soddisfa la proprietà max-heap con al più una violazione in $(A[i], A[\text{Parent}(i)])$.

Insert: heapsize := heapsize + 1, $A[\text{heapsize}] := -\infty$, heapIncreaseKey($A, \text{heapsize}, \text{key}$).

Varianti e Problemi Tipici

Heap d -ario (es. Ternario $d = 3$): nodi interni hanno d figli.

- **Figli (1-based):** $d(i - 1) + 2$ fino a $d(i - 1) + d + 1$
- **Padre di j :** $\lfloor (j + d - 2)/d \rfloor$
- Altezza $\approx \log_d n$. Extract-Max costa $O(d \log_d n)$, Insert costa $O(\log_d n)$.

Trovare il k -esimo elemento minore:

1. Ordinamento: $O(n \log n)$.
2. Costruzione min-heap $O(n)$ e k Extract-Min: $O(n + k \log n)$.
3. Max-heap di dim k : Inserisci primi k el., per i successivi se $< \text{Max}$, fai Extract-Max e Insert. Tempo $O(n \log k)$.

Liste, Pile, Code, Alberi Binari

Array

Memorizzazione contigua, accesso diretto per indice in $O(1)$.

indirizzo $A[i]$ = indirizzo($A[1]$) + $(i - 1) \cdot \text{dim. elemento}$

Access(A, i)	$O(1)$
Search(A, x)	$O(n)$ — scansione lineare
Insert(A, i, x)	$O(n)$ — shift elementi a dx
Delete(A, i)	$O(n)$ — shift elementi a sx
Insert in coda: $O(1)$ se c'è spazio.	

Liste concatenate

Nodi collegati da puntatori, non contigui in memoria.

- **Semplice:** key, next · **Doppia:** key, next, prev

Operazioni (lista doppia con $L.\text{head}$)

Ricerca — $O(n)$: scorri da $L.\text{head}$ fino a trovare chiave k o NIL.

```
listSearch(L, k):
    x = L.head
    while x != NIL && x.key != k:
        x = x.next
    return x
```

Inserimento in testa — $O(1)$: listInsert(L, x):

```
x.next = L.head
if L.head != NIL: L.head.prev = x
L.head = x; x.prev = NIL
```

Cancellazione (dato il puntatore) — $O(1)$: listDelete(L, x):

```
if x.prev != NIL: x.prev.next = x.next
else: L.head = x.next
if x.next != NIL: x.next.prev = x.prev
```

Varianti

- **Circolare:** ultimo nodo punta al primo
- **Con sentinella** ($L.\text{nil}$): nodo fittizio, elimina tutti i casi limite. $L.\text{nil}.\text{next} = \text{testa}$, $L.\text{nil}.\text{prev} = \text{coda}$. Lista vuota: $L.\text{nil}.\text{next} == L.\text{nil}$.

Con sentinella, cancellazione diventa solo: $x.\text{prev}.\text{next} = x.\text{next}$; $x.\text{next}.\text{prev} = x.\text{prev}$

Array vs Lista:

Operazione	Array	Lista
Accesso indice	$O(1)$	$O(n)$
Ricerca	$O(n)$	$O(n)$
Insert in testa	$O(n)$	$O(1)$
Delete (dato ptr)	$O(n)$	$O(1)$
Località cache	Alta	Scarsa

Pile (Stack) — LIFO

Ultimo inserito, primo rimosso. Operazioni su cima (top).

Push(S, x) $O(1)$ · Pop(S) $O(1)$ · Top(S) $O(1)$ · IsEmpty(S) $O(1)$

Su array $S[1..n]$ con $S.\text{top}$ (vuota se $S.\text{top} == \emptyset$):

```
push(S, x):
    S.top = S.top + 1; S[S.top] = x
pop(S):
    S.top = S.top - 1; return S[S.top + 1]
```

Su lista: Push = insert in testa, Pop = delete in testa.

Applicazioni: call stack, backtracking, bilanciamento parentesi, undo/redo, valutazione espressioni postfixe.

Code (Queue) — FIFO

Primo inserito, primo rimosso. Enqueue in coda, Dequeue in testa.

Enqueue(Q, x) $O(1)$ · Dequeue(Q) $O(1)$ · Front(Q) $O(1)$ · IsEmpty(Q) $O(1)$

Array circolare $Q[1..n]$ con $Q.\text{head}$, $Q.\text{tail}$:

- Vuota: $Q.\text{head} == Q.\text{tail}$
 - Piena: $(Q.\text{tail} \bmod n) + 1 == Q.\text{head}$
 - Capacità effettiva: $n - 1$ (una cella sacrificata per distinguere vuota/piena)
 - Avanzamento: $i := (i \bmod n) + 1$
- ```
enqueue(Q, x):
 Q[Q.tail] = x
 Q.tail = (Q.tail mod Q.length) + 1
dequeue(Q):
```

```
x = Q[Q.head]
Q.head = (Q.head mod Q.length) + 1
return x
```

Applicazioni: scheduling round-robin, buffer I/O, BFS, gestione richieste.

### Deque (Double-Ended Queue)

Generalizza pila e coda: inserimenti e rimozioni da **entrambe** le estremità. Tutte le operazioni  $O(1)$ .

### Alberi binari — Definizioni

**Albero binario:** vuoto (NIL) oppure nodo con key, left, right, parent.

#### Terminologia:

- **Radice:** nodo senza padre ( $T.root$ )
- **Foglia:** nodo con entrambi i figli = NIL
- **Profondità:** #archi dalla radice al nodo
- **Altezza nodo:** #archi nel cammino più lungo verso una foglia
- **Altezza albero:** altezza della radice

**Proprietà:** albero binario di altezza  $h$ :

- Max nodi:  $2^{h+1} - 1$  · Max foglie:  $2^h$
- Se  $n$  nodi:  $h \geq \lceil \log_2(n+1) \rceil - 1$
- **Completo:**  $2^{h+1} - 1$  nodi · **Degenerare:**  $h+1$  nodi (= lista)

**Figlio sx - Fratello dx:** rappresenta alberi con #figli arbitrario usando 2 puntatori: left-child (primo figlio), right-sibling (fratello destro).

#### Visite — $\Theta(n)$

**Inorder (sx, radice, dx):**  $T(n) = T(k) + T(n-k-1) + d = \Theta(n)$

```
inorderTreeWalk(x):
 if x != NIL:
 inorderTreeWalk(x.left)
 visita(x)
 inorderTreeWalk(x.right)
```

**Preorder (radice, sx, dx) · Postorder (sx, dx, radice):** stessa struttura, cambia posizione di visita(x).

### Dizionari: ABR e Tabelle Hash

#### Insieme dinamico e Dizionario

**Insieme dinamico:** collezione modificabile con operazioni Search, Min, Max, Successor, Predecessor, Insert, Delete.

**Dizionario:** ADT con Search, Insert, Delete. Se supporta anche Min/Max/Succ/Pred: dizionario ordinato.

#### ABR (Albero Binario di Ricerca)

**Proprietà BST:** per ogni nodo  $x$ :

- ogni  $y$  nel sottoalbero sx:  $y.key < x.key$
- ogni  $y$  nel sottoalbero dx:  $y.key \geq x.key$

Proprietà **globale**, non solo rispetto ai figli diretti.

**Visita inorder di un BST** → chiavi in ordine non decrescente.

#### Operazioni — tutte $O(h)$

**Ricerca:** treeSearch(x, k):

```
while x != NIL && k != x.key:
 if k < x.key: x = x.left
 else: x = x.right
return x
```

**Minimo / Massimo:** treeMinimum(x): treeMaximum(x):

```
while x.left != NIL: while x.right != NIL:
 x = x.left x = x.right
return x return x
```

**Successore:** (1) se  $x$  ha figlio dx: minimo del sottoalbero dx. (2) altrimenti: risali finché non sei un figlio sx.

```
treeSuccessor(x):
 if x.right != NIL: return treeMinimum(x.right)
 y = x.parent
 while y != NIL && x == y.right:
 x = y; y = y.parent
 return y
```

**Predecessore:** simmetrico (massimo sottoalbero sx, oppure risali finché sei figlio dx).

#### Inserimento — $O(h)$

Scendi dalla radice confrontando le chiavi. Il trailing pointer  $y$  tiene traccia del padre. Il nuovo nodo diventa foglia.

```
treeInsert(T, z):
 y = NIL; x = T.root
 while x != NIL:
 y = x
 if z.key < x.key: x = x.left
 else: x = x.right
 z.parent = y
 if y == NIL: T.root = z
 elif z.key < y.key: y.left = z
 else: y.right = z
 z.left = NIL; z.right = NIL
```

#### Cancellazione — $O(h)$

Tre casi:

1.  $z$  **foglia:** rimuovi aggiornando puntatore del padre
2.  $z$  **ha 1 figlio:** collega figlio direttamente al padre di  $z$
3.  $z$  **ha 2 figli:** sostituisci  $z$  con il suo successore  $y$  (minimo sottoalbero dx, che non ha figlio sx)

**Transplant (ausiliaria):** sostituisce sottoalbero radicato in  $u$  con sottoalbero radicato in  $v$ .

```
transplant(T, u, v):
 if u.parent == NIL: T.root = v
 elif u == u.parent.left: u.parent.left = v
 else: u.parent.right = v
 if v != NIL: v.parent = u.parent
```

```
treeDelete(T, z):
 if z.left == NIL: transplant(T, z, z.right)
 elif z.right == NIL: transplant(T, z, z.left)
 else:
 y = treeMinimum(z.right) // successore
```

```
if y.parent != z:
 transplant(T, y, y.right)
 y.right = z.right; y.right.parent = y
 transplant(T, z, y)
 y.left = z.left; y.left.parent = y
```

#### Complessità BST

| Caso                  | Altezza $h$      | Operazioni  |
|-----------------------|------------------|-------------|
| Peggior (degenere)    | $n - 1$          | $O(n)$      |
| Migliore (bilanciato) | $\Theta(\log n)$ | $O(\log n)$ |
| Medio (casuale)       | $O(\log n)$      | $O(\log n)$ |

Per  $O(\log n)$  garantito nel caso pessimo: AVL, Red-Black Tree (rotazioni).

#### Tabelle Hash

##### Indirizzamento diretto

Universo  $U = \{0, \dots, m-1\}$  piccolo → array  $T[0..m-1]$  dove  $T[k]$  contiene l'elemento con chiave  $k$ .

Search, Insert, Delete: tutte  $O(1)$ . Spazio:  $O(|U|)$  (sprecato se  $n \ll |U|$ ).

##### Tavola hash con concatenamento

**Funzione hash:**  $h: U \rightarrow \{0, \dots, m-1\}$ , con  $m \ll |U|$ .

**Collisione:**  $k_1 \neq k_2$  ma  $h(k_1) = h(k_2)$  (inevitabile se  $|U| > m$ ).

**Concatenamento:** ogni cella  $T[j]$  contiene una lista concatenata di tutti gli elementi con hash  $j$ .

- **Insert:** in testa alla lista  $T[h(k)] - O(1)$
- **Search:** scansione della lista  $T[h(k)]$
- **Delete:** rimuovi dalla lista (con lista doppia:  $O(1)$ )

#### Analisi

**Fattore di carico:**  $\alpha = n/m$  (media elementi per cella)

**Ipotesi hashing uniforme semplice:** ogni chiave ha uguale probabilità di essere mappata in ciascuna delle  $m$  celle.

**Caso pessimo:** tutte le  $n$  chiavi nella stessa cella →  $\Theta(n)$

**Caso medio (hashing uniforme):**

- Ricerca senza successo:  $\Theta(1 + \alpha)$
- Ricerca con successo:  $\Theta(1 + \alpha)$

Se  $n = O(m)$  allora  $\alpha = O(1)$  e tutte le operazioni sono  $O(1)$  in media.

#### Metodo della divisione

$$h(k) = k \bmod m$$

#### Scelta di $m$ :

- **Evitare  $m = 2^p$ :** hash dipende solo dagli ultimi  $p$  bit
- **Evitare  $m = 2^p - 1$ :** permutazioni di stringhe danno stesso hash
- **Buona scelta:** numero primo non vicino a potenze di 2

Chiavi non numeriche → convertire in  $\mathbb{N}$  (es. stringa in base 128 tramite codici ASCII).

#### Indirizzamento aperto

Tutti gli elementi in array  $T$  (nessuna lista esterna,  $\alpha \leq 1$ ). Si ispezionano sequenze di celle usando funzione hash estesa:  $h(k, i)$  con  $i = 0, \dots, m-1$ .

- **Lineare:**  $h(k, i) = (h'(k) + i) \bmod m$  (soffre di clustering primario).
- **Quadratica:**  $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$  (clustering secondario).
- **Doppio hash:**  $h(k, i) = (h_1(k) + i h_2(k)) \bmod m$ .  $h_2(k)$  e  $m$  devono essere coprimi.

**Ricerca/Inserimento:** ispeziona sequenza finché non trovi  $k$  o cella vuota.

**Cancellazione:** marca come DELETED per non interrompere la ricerca.

#### Confronto realizzazioni di dizionario

| Op.       | Indir. dir. | Hash (medio)    | ABR (caso peg.) |
|-----------|-------------|-----------------|-----------------|
| Search    | $O(1)$      | $O(1 + \alpha)$ | $O(h)$          |
| Insert    | $O(1)$      | $O(1)$          | $O(h)$          |
| Delete    | $O(1)$      | $O(1)$          | $O(h)$          |
| Min/Max   | $O(m)$      | —               | $O(h)$          |
| Succ/Pred | $O(m)$      | —               | $O(h)$          |
| Spazio    | $O( U )$    | $O(m + n)$      | $O(n)$          |

Hash: migliore per Search/Insert/Delete, ma no Min/Max/Succ/Pred.  
ABR: migliore per dizionario **ordinato**.

### Problem Solving: Alberi e Array

#### Pattern ricorsivi su alberi binari

Struttura base: se nodo è NIL → caso base; altrimenti elabora ricorsivamente figli e combina risultati.

#### Altezza di un albero

```
int treeHeight(Node x):
 if x == NIL: return -1
 lh = treeHeight(x.left)
 rh = treeHeight(x.right)
 return 1 + max(lh, rh)
```

Complessità:  $\Theta(n)$  — visita ogni nodo una volta.

#### Contare i nodi

```
int countNodes(Node x):
 if x == NIL: return 0
 return 1 + countNodes(x.left) + countNodes(x.right)
```

Contare le foglie

```
int countLeaves(Node x):
 if x == NIL: return 0
 if x.left == NIL && x.right == NIL: return 1
 return countLeaves(x.left) + countLeaves(x.right)
```

Verifica proprietà BST

Non basta confrontare con i figli diretti: servono limiti globali.

```
bool isBST(Node x, int min, int max):
 if x == NIL: return true
 if x.key < min || x.key > max: return false
 return isBST(x.left, min, x.key - 1)
 && isBST(x.right, x.key, max)
```

Chiamata iniziale: isBST(T.root, -∞, +∞). Complessità:  $O(n)$ .

Invertire un Albero (Specchio)

Scambia figlio sx e dx per ogni nodo. Se ABR, si inverte la proprietà.

```
invertTree(Node x):
 if x == NIL: return
 swap(x.left, x.right)
 invertTree(x.left); invertTree(x.right)
```

Verifica 1-Bilanciamento

Ritorna altezza se 1-bilanciato, altrimenti  $-1$ . Complessità  $O(n)$ .

```
int checkBalance(Node x):
 if x == NIL: return 0
 lh = checkBalance(x.left)
 rh = checkBalance(x.right)
 if lh == -1 || rh == -1 || abs(lh - rh) > 1: return -1
 return 1 + max(lh, rh)
```

Pattern su array

Ricerca lineare e sentinella

Ricerca base:  $O(n)$ . Con sentinella ( $A[n+1] := k$ ) si elimina il controllo sul bound, riducendo il fattore costante.

Due puntatori (two pointers)

Utile per array ordinati o per trovare coppie con proprietà.

```
// Coppia con somma target in array ordinato
bool twoSum(int[] A, int n, int target):
 i = 1; j = n
 while i < j:
 s = A[i] + A[j]
 if s == target: return true
 elif s < target: i = i + 1
 else: j = j - 1
 return false
```

Complessità:  $O(n)$ .

Ricerca binaria —  $O(\log n)$

Prerequisito: array ordinato. Dimezza lo spazio di ricerca ad ogni passo.

```
int binarySearch(int[] A, int lo, int hi, int k):
 while lo <= hi:
 mid = (lo + hi) / 2
 if A[mid] == k: return mid
 elif A[mid] < k: lo = mid + 1
 else: hi = mid - 1
 return -1 // non trovato
```

Ricorrenza:  $T(n) = T(n/2) + O(1) = O(\log n)$ .

Complessità algoritmi ricorsivi

Master Theorem (Teorema Principale)

Risolve ricorrenze:  $T(n) = aT(n/b) + f(n)$  con  $a \geq 1, b > 1$ .

Sia  $c = \log_b a$ , confrontiamo  $f(n)$  con  $n^c$ :

1. **Caso 1 (Dominato dalle foglie):** Se  $f(n) = O(n^{c-\epsilon})$  per  $\epsilon > 0 \rightarrow T(n) = \Theta(n^c)$

2. **Caso 2 (Bilanciato - esteso):** Se  $f(n) = \Theta(n^c \log^k n)$  per  $k \geq 0 \rightarrow T(n) = \Theta(n^c \log^{k+1} n)$  (Es: se  $f(n) = \Theta(n^c)$  allora  $k = 0 \rightarrow T(n) = \Theta(n^c \log n)$ )

3. **Caso 3 (Dominato dalla radice):** Se  $f(n) = \Omega(n^{c+\epsilon})$  per  $\epsilon > 0$  E se vale la condizione di regolarità:  $af(n/b) \leq df(n)$  per  $d < 1 \rightarrow T(n) = \Theta(f(n))$

Guida all'esame:

- 1. Calcola  $c = \log_b a$  e scrivi  $n^c$ .
- 2. Confronta asintoticamente  $n^c$  con  $f(n)$ . Se uno cresce "polinomialmente più veloce", ricadiamo nel Caso 1 o 3. Se hanno la stessa crescita (a meno di log), è il Caso 2.

Esempi pratici:

- $T(n) = 2T(n/2) + \Theta(n)$ :  $a = 2, b = 2 \Rightarrow c = 1$ .  $n^1$  vs  $n^1$ . Caso 2 ( $k = 0$ )  $\rightarrow \Theta(n \log n)$
- $T(n) = T(n/2) + \Theta(1)$ :  $a = 1, b = 2 \Rightarrow c = 0$ .  $n^0$  vs 1. Caso 2 ( $k = 0$ )  $\rightarrow \Theta(\log n)$
- $T(n) = 4T(n/2) + \Theta(n)$ :  $a = 4, b = 2 \Rightarrow c = 2$ .  $n^2$  vs  $n^1$ . Caso 1  $\rightarrow \Theta(n^2)$
- $T(n) = 3T(n/4) + n \log n$ :  $a = 3, b = 4 \Rightarrow c \approx 0.79$ .  $n^{0.79}$  vs  $n^1$ . Caso 3  $\rightarrow \Theta(n \log n)$
- $T(n) = 2T(n/2) + n \log n$ :  $a = 2, b = 2 \Rightarrow c = 1$ .  $n^1$  vs  $n^1 \log n$ . Caso 2 ( $k = 1$ )  $\rightarrow \Theta(n \log^2 n)$

Albero di ricorsione (metodo iterativo)

Espandere la ricorrenza fino al caso base, sommare i costi per livello. Utile quando il Master Theorem non si applica.

Strategie generali

1. **Identifica la ricorrenza:** numero chiamate ricorsive ( $a$ ), dimensione sottoproblemi ( $n/b$ ), costo combinazione ( $f(n)$ )

2. **Per alberi:** pensa alla struttura ricorsiva — caso base (NIL/foglia), combina risultati figli

3. **Per array:** considera approcci in-place (due puntatori, partizione), oppure divide-et-impera

4. **Verifica correttezza:** invariante di ciclo (inizializzazione, mantenimento, terminazione) o induzione

5. **Complessità:** Master Theorem per D&I, somma costi per iterativi

Ordinamenti Lineari

Per scendere sotto il limite  $\Omega(n \log n)$  degli ordinamenti basati su confronti, si sfruttano proprietà dei dati (es. interi limitati).

Counting Sort —  $\Theta(n + k)$

Ordina  $n$  interi nell'intervallo  $[0, k]$ . Usa gli elementi come indici. È **stabile**.

Fasi:

1. **Frequenze:** Conta occorrenze in  $C[0..k]$ .

2. **Cumulate:**  $C[i] += C[i-1]$  (posizioni finali).

3. **Costruzione:** Scorri A al contrario per stabilità: metti  $A[j]$  in  $B[C[A[j]]]$  e decrementa  $C[A[j]]$ .

```
countingSort(A, B, k):
 let C[0..k] = 0
 for j = 1 to A.length: C[A[j]]++
 for i = 1 to k: C[i] += C[i - 1]
 for j = A.length downto 1:
 B[C[A[j]]] = A[j]
 C[A[j]]--
```

**Tempo:**  $\Theta(n + k)$ . Se  $k = O(n)$ , il tempo è  $\Theta(n)$ . **Spazio:**  $O(n + k)$  (non in-place).

Radix Sort —  $\Theta(d(n + k))$

Ordina numeri di  $d$  cifre partendo dalla meno significativa (LSD, Least Significant Digit) alla più significativa.

```
radixSort(A, d):
 for i = 1 to d:
 usa ordinamento stabile (es. Counting Sort) su cifra i-esima
```

Analisi e Note:

- **Perché LSD:** Se due numeri hanno la stessa cifra su colonna  $i$ , l'algoritmo stabile preserva il loro ordine relativo, determinato in modo corretto dai passi precedenti (cifre meno significative).
- **Tempo:**  $d$  passate di Counting Sort. Totale  $\Theta(d(n + k))$  dove  $k$  è la base (es. 10, o  $2^r$ ).
- **Ottimizzazione bit:** Un intero di  $b$  bit si può spezzare in  $d = \lceil b/r \rceil$  cifre da  $r$  bit. Con  $r \approx \log n$ , il tempo diventa  $O(bn/\log n)$ .